Shogi4: a validated strong-solution engine for 4×4 drop-shogi, and a de-risked path to the frontier

Brian Liou · June 2026

Drop-shogi is the live frontier of strongly-solved games. Dōbutsu Shōgi (3×4) is the largest one solved — Tanaka strongly solved it in 2009. I reproduced that solution from scratch and validated it position-by-position against `clausecker/dobutsu`, then built and validated the **complete strong-solution engine for the next rung up, Shogi4** (a public-domain 4×4 drop-shogi), and de-risked its full distributed solve to the point where only the compute run remains. The rung after that, Micro Shogi (4×5), is open.

Game	Board	Reachable positions	Status
Dōbutsu Shōgi	3×4	2.47×10^{8}	solved 2009 (Tanaka); reproduced + validated vs `clausecker/dobutsu`
Shogi4	**4×4**	**~3×10^{13}**	**engine complete + oracle-validated; 51.5M partial solve; full run calibrated**
Micro Shogi	4×5	~5×10^{14}	open frontier

Why drop-shogi is the interesting regime

Drops break the chess endgame-tablebase recipe. In chess, captures strictly reduce material, so endgame tables solve a clean bottom-up DAG of material buckets. Shogi drops let captured pieces re-enter play, so material is conserved and the position graph collapses into one large strongly-connected cycle. The standard recipe doesn't apply unmodified. Dōbutsu hid this — it fit in RAM and was solved whole. Shogi4 forces the real problem: an external-memory, distributed retrograde computation, at the smallest scale where every design decision is still laptop-testable but the full run genuinely needs a cluster.

Shogi4 has never been solved — no published game value exists. It's public domain (Oca Studios), so the solution is free to compute, publish, and redistribute.

The artifact

Rules, recovered. Shogi4's authoritative rules were unreachable — the publisher's server is dead and the print-and-play rulesheet was never archived. I recovered them by decompiling the delisted official app (a Python/SDL build whose payload was hidden as a fake `.mp3`); its move-generation logic *is* the authoritative ruleset, including a "friendly-jump" mechanic and capture-flip promotion.

A from-scratch solver, validated against an independent oracle. The Rust engine is move generation, a dense position-ranking bijection, un-move generation, and push-based retrograde. Every component is cross-checked against a separate Python reference (a port of the decompiled app):

- **perft** from the start matches to the digit (`8 · 64 · 626 · 6,304 · 68,723 · 769,014`), plus a 4,000-position golden move-list diff with zero mismatches.
- **Three independent solvers** (two algorithms in Python, one in Rust) agree to the unit on every sub-game, up to 1,164,704 positions.

- The **dense rank** is a verified bijection; its domain `N = 410,297,064,507,360` equals exactly 2× the independently-computed state-space count.

The numbers. The all-arrangements upper bound is **205,148,532,253,680** — exact, computed by an enumerator that reproduces Tanaka's published Dōbutsu figure (`1,567,925,964`) and its full breakdown to the digit. Reachable from the start is ~3×10^{13}.

A 51-million-position partial solve. The largest in-RAM solve to date — a 51,461,568-position sub-game — solved and consistency-audited (W 80.3% / L 18.9% / D 0.8%).

The full solve, de-risked for $0

The complete solve is an external-memory, sharded, push-based retrograde over a ~50–100 TB value array: BSP supersteps, a predecessor-update shuffle, SCC-staged scheduling. I validated the design on a single laptop, against the in-RAM oracle, before committing a dollar of hardware:

- **Confluence.** The sharded BSP solver reproduces the oracle's result for *every* shard count (1, 4, 16, 64). Parallelization cannot change the answer — the algorithm is a monotone, order-independent fixpoint.
- **Cheap verification.** A single-pass consistency audit (every value must be the minimax of its children) certifies the tablebase at **~55% of the solve cost**, and catches every injected corruption. A wrong value fails its own local check, so silent corruption can't pass.
- **Failure recovery.** Crash plus deterministic replay reproduces each superstep byte-for-byte and yields a result identical to the crash-free run. Building this surfaced and fixed a latent nondeterminism, making the whole solver reproducible — "a shard is a checkpoint plus a deterministic recompute."
- **Calibration.** A 51.5M-position run measured per-edge cost rising from 396 to 633 ns as the working set grew past cache, which is the external-memory cost risk appearing in data rather than in a guess.

Cost, staged and self-corrected

The full Shogi4 solve is ~50–100 TB and ~60–190 core-years (≈$5–15k bare-metal), with an exact 2× left–right symmetry fold (proven, not assumed). I corrected my own cost estimate twice during this work: first an order-of-magnitude error in the raw state-space count, then the realization that a dense-rank solver's footprint is the *arrangement domain*, not the reachable count (~13× larger). Both corrections sit in the record. The plan is gated: a ~$40 partial-bucket run calibrates the real per-edge cost on the target architecture before any large commit, and the SCC staging plus per-bucket audits mean cost and correctness surprises surface on cheap, early buckets — not after a multi-week run.

The same ladder continues to Micro Shogi (~19× larger, the open frontier), whose published estimates I've corrected the same way.

Lineage

- **Tanaka (2009)** — strong solution of Dōbutsu Shōgi; reproduced here from scratch.
- **`clausecker/dobutsu`** — the reference tablebase this work validates against, position-by-position.
- **Schaeffer et al. (2007)** — the solution of Checkers via retrograde endgame databases at scale: the same technique, two rungs up.

About

Solo and independent. The full source — engine, validation harnesses, and the research ledger that records every number with its provenance and every self-correction — is reproducible end-to-end and available on request. The natural next step is the distributed run itself, on a cluster: the pipeline is validated against an oracle, the cost is calibrated, and the distributed design is proven correct under sharding and worker failure. I'd welcome collaboration with anyone working in this lineage.